

Autonomous API Discovery & Integration Engine

A first-principles engineering report

Version 1.1 · 14 September 2026 · commit `5966328`

This document explains the project from the ground up. It assumes no prior knowledge of APIs, search engines, databases, or language models. Every technical term is defined the first time it appears.

It is deliberately not a sales document. The most useful sections are the ones recording decisions that turned out to be **wrong**, because those are where the reasoning is visible. A report that lists only the things that worked teaches nothing about how the work was actually done.

Part I assumes no background at all and builds the vocabulary from what an API call is up to what a DAG is; skip it if that is familiar. **§2.5 is the whole system working on one real goal, start to finish** — a good place to begin if you would rather see the thing run before reading how it is built.

Part I — Concepts from scratch

Nothing in this part is specific to the project. If you already know what an API and a JSON Schema are, skip to Part II.

1.1 What a program does when it "talks to" another program

Software rarely works alone. A shop's website needs to charge a card, so it asks a payment company to do it. A project tracker needs to notify a team, so it asks a chat company. The mechanism for one program to ask another program to do something, over the internet, is an **API** — an Application Programming Interface.

Concretely, an API call is an HTTP request: the same protocol your browser uses to fetch a web page, but the reply is structured data rather than a page to look at.

A request has four parts that matter here.

Part	What it is	Example
Method	The verb — what kind of action	GET (fetch), POST (create), DELETE (remove)
URL	The address of the thing you are acting on	<code>https://slack.com/api/chat.postMessage</code>
Headers	Metadata travelling alongside, including credentials	<code>Authorization: Bearer xoxb-...</code>
Body	The data you are sending, usually JSON	<code>{"channel": "#eng", "text": "hello"}</code>

The reply has a **status code** — a three-digit number where `2xx` conventionally means success, `4xx` means you asked wrongly, and `5xx` means the other side broke — and usually a JSON body.

JSON is the standard text format for structured data: nested objects of key/value pairs. `{"title": "Bug in login", "user": {"login": "alex"}}` is a JSON object containing a string and a nested object.

1.2 Endpoints

An API is not one thing you call. It is a *catalogue* of operations, each called an **endpoint**. GitHub has roughly 845 of them; one creates an issue, another lists commits, another deletes a repository.

An endpoint is identified by a **method** plus a **path**:

```
POST /repos/{owner}/{repo}/issues
```

The braces are blanks you fill in — **path parameters**. Other values can travel as **query parameters** (after a `?` in the URL), as **headers**, or in the **body**.

1.3 Schemas: the shape of data

If you are going to move data from one API into another automatically, you need to know the *shape* of what comes out and the shape of what goes in. That description is a **schema**.

JSON Schema is the standard way to write one:

```
{
  "type": "object",
  "required": ["title"],
  "properties": {
    "title": {"type": "string"},
    "body": {"type": "string"},
    "user": {"type": "object", "properties": {"login": {"type": "string"}}}
  }
}
```

That says: an object; it must have `title`; `title` is text; there may be a nested `user` object containing `login`.

Two ideas from this matter enormously later:

- **required** — the fields that must be present. This is what lets a machine say "this request is missing something" *before* sending it.
- **Nesting** — `user.login` is a path *into* the shape. Machinery that automatically connects two APIs must be able to walk such paths.

1.4 OpenAPI: the machine-readable manual

Documentation written for humans cannot be processed by a program. So an industry convention emerged: publish a single file describing the entire API in a fixed format. That format is **OpenAPI** (formerly **Swagger**).

One OpenAPI document lists every endpoint, every parameter, every schema, and how to authenticate. It exists precisely so that *programs* can understand an API.

This is the raw material the whole project runs on. Without it, there would be nothing to validate against, and the central claim would be impossible.

Two complications that cost real effort later:

1. **There are two incompatible versions in the wild** — Swagger 2.0 and OpenAPI 3.x — which disagree about where the server address lives, how the request body is described, and how authentication is declared.
2. **Specifications use references.** Rather than repeating a schema, a document says `{"$ref": "#/components/schemas/Event"}`, meaning "the thing defined over there". Following those is called **reference resolution**, and doing it safely is harder than it sounds (§5.3).

1.5 Authentication

Most APIs will not act for an anonymous caller. Proving who you are is **authentication**, and there are a handful of common schemes.

Scheme	How it works	Can you paste it?
API key	A secret string in a header or query parameter	Yes
Bearer token	Authorization: Bearer <token>	Yes
Basic	A username and password, encoded	Yes
OAuth 2.0	You are redirected to the provider, approve, and the application receives a token	No — it must be performed

The last one matters. **OAuth 2.0 is a conversation, not a string.** The application sends you to Google, you approve, Google hands back a short-lived code, and the application exchanges that code — using a secret only the server knows — for an access token and a **refresh token**. Access tokens expire within about an hour; the refresh token is what silently renews them.

1.6 Searching text: what BM25 is

Given a question in words and a large pile of documents, which documents are relevant? The classical answer is **lexical retrieval**: match words, and weight each match by how *rare* the word is.

The intuition: in a search for "slack message", the word "message" appears in thousands of documents and tells you almost nothing. "Slack" appears in few and tells you a great deal. A document should score higher for matching a rare word than a common one.

BM25 is the refined form of that idea, and has been the backbone of search engines for decades. It weighs three things:

- **Term frequency** — how often the word appears in this document, with diminishing returns
- **Inverse document frequency** — how rare the word is across everything
- **Length normalisation** — so a long document does not win just by being long

It is arithmetic, not AI. It is extremely fast, and it is *explainable*: you can say exactly why a result ranked where it did.

Its fatal weakness is vocabulary. Search "notify my team" and BM25 finds nothing in a document that says "post a message to a channel" — no shared words, identical meaning.

1.7 Embeddings: searching by meaning

An **embedding** is a list of numbers — a *vector* — that represents the meaning of a piece of text. A model is trained so that texts with similar meanings get similar vectors, even with no words in common.

"Similar" is measured by **cosine similarity**: the angle between two vectors. Identical direction scores 1.0, unrelated scores near 0.

So: convert every document to a vector once, convert the question to a vector at search time, and return the nearest. This solves the vocabulary problem — but introduces its own failure mode, being vague. Semantic search will happily return something *thematically* close when you needed an exact token like `chat.postMessage`.

1.8 Combining both: rank fusion

Lexical and semantic search fail in different, complementary ways. So use both and merge the rankings.

Merging by *score* is a trap: BM25 scores and cosine similarities are on incomparable scales, so combining them requires a normalisation step that is itself a tuning problem.

Reciprocal Rank Fusion (RRF) sidesteps this by ignoring scores entirely and using only *position*:

$$\text{score}(\text{document}) = \sum \frac{1}{k + \text{rank_in_that_list}}$$

k is a constant (60 here) that dampens the influence of the very top ranks. A document ranked #2 by both methods beats one ranked #1 by a single method and ignored by the other. One parameter, no scale problem.

1.9 How you measure a search engine

Opinions about search quality are worthless. The measurements are:

You write a **labelled query set** — questions paired with the answers you know to be correct — then score the engine against it.

Metric	Plain meaning
Precision@5	Of the top 5 results, what fraction were correct
Recall@20	Of all correct answers that exist, what fraction reached the top 20
MRR	Average of $1/(\text{rank of first correct answer})$. 1.0 = always first
NDCG@10	Ranking quality with a logarithmic discount for lower positions, normalised so queries with different numbers of answers compare fairly

Latency percentiles matter too. p_{50} is the median; p_{95} is the value 95% of requests come in under. Averages hide the slow tail, which is what users actually notice.

1.10 Large language models, and what they are bad at

A **large language model** (LLM) predicts likely text. Given a description of a task it produces a plausible continuation. It is excellent at turning messy human phrasing into structure, and at judging semantic fit.

It has one property that dominates every design decision in this project:

It produces plausible text, not true text, and the two are indistinguishable from the output alone.

Asked for a GitHub field name it will confidently answer `issue.author`, because that is what a sensible API *would* call it. The real field is `issue.user.login`. The model states both kinds of answer with identical confidence.

This is usually called **hallucination**. The engineering response is not to hope for a better model. It is to build a system in which a fabricated answer *cannot take effect*.

1.11 The vocabulary of reliability

The final concepts, needed for Part IV.

- **Idempotency** — an operation that can be repeated without changing the result beyond the first time. Sending a message is *not* naturally idempotent: do it twice and two messages exist. An **idempotency key** is a unique value sent with the request so the provider recognises a repeat and ignores it.
- **At-least-once delivery** — a queue guarantees a job is delivered, possibly more than once. Combined with idempotency, that is enough. Pretending to have exactly-once delivery is a well-known way to be wrong.
- **Checkpointing** — writing progress durably so a crash resumes rather than restarts.
- **Exponential backoff** — waiting longer after each failed retry, with random **jitter** so many clients do not retry in lockstep after a shared outage.
- **SSRF** (Server-Side Request Forgery) — an attack where a system that fetches URLs on your behalf is pointed at internal infrastructure. Cloud metadata services at `169.254.169.254` hand out credentials to anything inside the machine that asks.

1.12 Workflows, steps, and why the shape is a graph

The last piece of vocabulary, and the one the whole of Part III is built on.

A **workflow** is an automation written down as data rather than as code. It is a list of **steps** (also called **nodes**), plus a record of which step feeds which. One step is the **trigger** — the thing that starts everything, such as "a GitHub issue was opened". The rest are **actions** — things to do, such as "post a Slack message".

Steps depend on each other, because a later step usually consumes an earlier step's output. Draw those dependencies as arrows and you get a **graph**:

```
[trigger]  GitHub: issue opened
          |
          |  issue.title, issue.user.login
          v
[action]   Slack: post message
```

That shape has a name: a **directed acyclic graph**, universally shortened to **DAG**. Three words, each doing work:

- **Graph** — things connected by arrows.
- **Directed** — the arrows have a direction. Output flows one way.
- **Acyclic** — no loops. You cannot follow the arrows and arrive back where you started.

The acyclic part is not a technicality. A cycle means step A waits for step B while step B waits for step A, so the workflow can never finish and nothing can say what its output is. Rejecting cycles is check #1 of the seven in §3.5, and it is checked structurally rather than discovered at runtime by hanging.

What "compiling" means here

The model does not emit runnable code. It emits a **plan** — a description of the intended workflow. That plan is then checked and turned into a **compiled intermediate representation**, usually shortened to **IR**.

"Intermediate" because it sits between the two ends: more precise than the English sentence, but not yet the actual HTTP requests. It is a fully resolved, verified description — every endpoint confirmed to exist, every field mapped to a real source, every type checked — stored as data.

The reason for the intermediate step is that it separates *deciding* from *doing*. Everything uncertain happens before the IR exists. Once it does, execution is mechanical, and the same IR run twice does the same thing. §4.5 covers the reasoning in full.

Triggers, webhooks, queues and workers

Four operational words the later parts use freely:

- A **webhook** is how one service tells another that something happened. Instead of you asking GitHub "any new issues?" every few seconds, GitHub sends an HTTP request *to you* the moment an issue opens. That request's body is the **payload**, and it is often large — the GitHub issue payload runs to about 200 lines, which is why finding the right field in it is real work.
- A **queue** is a waiting list of jobs. Work is put on the queue rather than done immediately, so a slow or failing job does not block whatever accepted it.
- A **worker** is the process that takes jobs off the queue and runs them. Splitting the two means the part that receives the webhook can answer instantly while the actual sending happens elsewhere, and a crash loses nothing because the job is still on the queue.
- A **dry run** executes the workflow with every outbound request suppressed. It proves the plan is wired correctly without sending anyone a message. Part VII counts dry runs separately from real executions for exactly that reason.

Part II — The problem, and what was built

2.1 The problem, concretely

"When someone files a bug on GitHub, put it in our team's Slack channel."

That sounds like five minutes. In practice:

1. Learn how GitHub can notify you — webhooks — which requires a publicly reachable server, so a tunnel for local testing
2. Subscribe to the right event, and discover the payload's `action` field must be filtered to `"opened"` or you get spammed
3. Find the fields in a 200-line payload. The title is at `issue.title`. The author is **not** `issue.author`; it is `issue.user.login`
4. Repeat for Slack: create an app, choose scopes, install, copy the token, and discover the bot must be invited to the channel or you get `not_in_channel`
5. Write the translation between the two shapes
6. Hit the trap: **Slack replies 200 OK with `{"ok": false}`** when it fails, so checking the status code alone reports success while doing nothing
7. Handle the questions you did not ask: what if Slack is down for 20 seconds? What if the program crashes after sending but before recording that it sent?

Honest cost: one to two days, for a disposable program that does one thing. A normal organisation runs thirty to fifty of these. Every one has the same shape and different details, so nothing is reusable.

2.2 Why existing answers are insufficient

Write it yourself. Total control, days per automation, maintained forever.

Zapier / Make / n8n. Genuinely good — but a human hand-built every connector. If your tool is not in the catalogue you are stuck; if it is, but the specific operation was not exposed, you are stuck. Internal APIs are never in the catalogue. **Their coverage is a hand-built list.**

Ask a language model to write the code. Fast, flexible, and disqualified by §1.10: it fabricates endpoints and field names with total confidence, and you discover this at runtime. It also hands you *code*, leaving deployment, monitoring and failure handling entirely to you.

2.3 The thesis

LLMs propose; deterministic systems validate and execute.

The model may *suggest* that GitHub's issue endpoint and Slack's message endpoint are the right pair, and that `issue.title` should flow into `text`. It may never *act* on that suggestion directly.

Between the model and the outside world sits a validator that re-checks every claim against a database built from real OpenAPI documents. Does this endpoint exist? Does the method match? Are the required fields supplied? Do the types line up? If any check fails the workflow is rejected — the model does not get to argue. If all pass, the executor builds the request **from the database record**, never from anything the model wrote.

The analogy that captures it: a very fast, very confident intern who has read a lot of documentation from memory, given a supervisor with the actual manual open who checks every claim before it reaches production.

And note *why* checking is possible at all. The supervisor is not judging whether the code is elegant. They are answering mechanical questions — does this exist, is this field a string — which have definite answers. **That is what OpenAPI buys you.**

2.4 What exists today

Providers indexed	120
Distinct APIs	270
Endpoints	15,504
Endpoints flagged destructive	1,872
Labelled benchmark queries	30
Automated tests	278
Web interface	5 pages

The whole backend runs on SQLite with no external services, no Docker and no container runtime.

2.5 One goal, end to end

Parts III to V take the system apart module by module. Before that, here is the whole thing working, on the goal from §2.1 — the one that costs a developer one to two days by hand.

"When someone files a bug on GitHub, put it in our team's Slack channel."

Step 1 — The sentence becomes a typed requirement

`agent/intent.py` sends the sentence to the language model and gets back structure rather than prose: there is one trigger, concerning issues, on GitHub; there is one action, sending a message, on Slack.

This is the model doing what it is genuinely good at — turning messy human phrasing into a shape. Nothing here is trusted yet. It is a *guess about what was meant*, and everything downstream treats it as one.

Step 2 — Search the corpus for candidate endpoints

That requirement becomes search queries against the indexed corpus of **15,504 endpoints**. Search runs in **hybrid** mode — keyword matching (BM25) and meaning-based matching (embeddings) combined, because §7.1 measures hybrid as better than either alone on every quality metric.

Only the **top 20** results per search survive. That cut-off matters more than it looks: the planner never sees rank 21, so anything below is invisible to the entire rest of the system. It is why §7.1 calls R@20 the operationally important number rather than P@5.

Step 3 — The model plans, but may only use what search returned

The retrieved candidates are handed to `agent/planner.py` **with their required parameters and their available response fields attached**, because a model cannot map a field it was never shown.

The model proposes a workflow: trigger on this GitHub endpoint, then call that Slack endpoint, taking the message text from the issue title.

Now the first guard applies. The model may name **only endpoint IDs that this request's searches actually returned**. Anything else is refused before the database is even consulted — and, deliberately, a real endpoint that exists in the database but was not retrieved for *this* goal is refused too. Existing is not enough. That closes the case where the model recalls a genuine endpoint from its training data and drops it in out of context (§3.6).

Step 4 — The compiler tries to reject the plan

Everything that survives goes to `workflow/compiler.py` and faces seven checks (§3.5): structure, existence, agreement, parameters, body, compatibility, policy.

Assume the model made the very mistake §1.10 predicts, and mapped the author from `issue.author` — a field that does not exist; the real path is `issue.user.login`. Check #6 catches it, and the refusal is a structured object, not a sentence:

```
{"code": "schema_incompatible", "node_id": "n3", "field": "message.text",  
  "expected": "string", "actual": "object",  
  "hint": "map issue.title into message.text"}
```

Step 5 — The refusal goes back to the model, at most three times

Those structured issues are fed back verbatim, the model revises, and the plan is re-checked. Three attempts maximum, then it gives up and says so rather than looping forever.

This is the point of the whole design. The model is allowed to be wrong, repeatedly, because being wrong is *cheap and contained* — a fabricated field name produces a rejection with a hint, not a broken automation discovered three weeks later. Measured cost: about **20 seconds** and **2,582 tokens** when the first attempt is accepted, 5,850 when one repair round is needed (§7.4).

Step 6 — A compiled IR, then execution

What survives is the compiled IR from §1.12: every endpoint verified, every field mapped, every type checked. Execution is now mechanical.

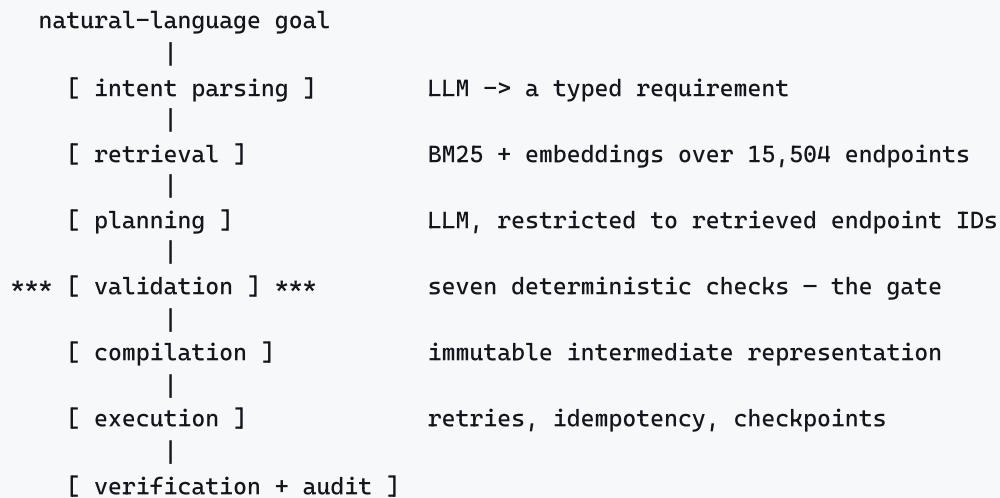
`execution/` resolves credentials (decrypting at the moment of use), checks the egress guard, and issues the request. Then the trap from §2.1 point 6: Slack answers `200 OK` with `{"ok": false}` when it has failed. The executor inspects the **response body**, not just the status code (§4.9), so that counts as a failure and is classified and retried rather than recorded as success.

What this actually proves, and what it does not

Being precise here, because "it works" is easy to overclaim. From §7.6: real requests have left the process, reached live services and returned success — **2 executions succeeded end to end, 4 live 200 responses**.

But **no authenticated call has ever been made against a live provider**, because no credential has ever been stored. The decrypt-apply-succeed path exists and is covered by tests against a mocked transport, never against the real thing. It is the cheapest remaining gap in the project, and §7.6 says so rather than letting the architecture diagram imply otherwise.

Part III — Architecture, module by module



Steps 1 and 3 are the language model. Step 4 is the gate. Everything else is ordinary careful engineering — and it is roughly 80% of the work, which is what makes this a systems project rather than a prompt.

3.1 `ingest/` — building the knowledge base

Responsible for: turning published OpenAPI documents into normalised database rows.

- `fetch.py` — downloads specifications from **APIs.guru**, a public directory of a few thousand real OpenAPI documents. Downloads run eight at a time, stream so an oversized document can be abandoned mid-transfer, and are cached on disk.
- `parse.py` — the hardest file in the project. Handles OpenAPI 3 *and* Swagger 2, resolves references, breaks reference cycles, extracts parameters with their required-ness, request and response schemas, and authentication schemes. Every document is processed inside error handling that logs and skips: **one broken specification must never abort a run.**
- `run.py` — orchestration and idempotent upsert.

3.2 `ids.py` — identifiers that cannot be guessed

Forty lines, and one of the more consequential decisions in the project.

Every identifier is a hash of its natural key:

```
endpoint_id("api_4d6f...", "POST", "/repos/{owner}/{repo}/issues") -> "ep_18da3450e690"
```

Two consequences:

1. **Re-ingesting updates rows instead of duplicating them**, because the same input always yields the same ID. Verified: running ingestion twice produces identical counts.

2. **The model cannot fabricate a plausible identifier.** Had IDs been readable — `github.create_issue` — a model could invent `github.delete_repo` and it would look legitimate. `ep_18da3450e690` is unguessable. **The ID format is itself part of the anti-hallucination defence.**

3.3 `db.py` — persistence

Thirteen tables: `providers`, `apis`, `endpoints`, `parameters`, `schemas`, `auth_schemes`, `ingest_runs`, `workflows`, `workflow_nodes`, `compiled_ir`, `credentials`, `executions`, `execution_events`, `node_results`, `jobs`, `credential_audit`.

SQLite in WAL mode so the API and the worker do not block each other. `execution_events` and `credential_audit` are append-only.

3.4 `search/` — retrieval

- `text.py` — pure functions, therefore exactly testable: path tokenisation, stopword removal, synonym expansion, and building a query expression that cannot be misread as search-engine syntax.
- `index.py` — builds a SQLite **FTS5** full-text index. FTS5 ships a genuine BM25 implementation, so lexical search needs no external service.
- `embeddings.py` — generates vectors locally via ONNX, stores them as one normalised `float32` matrix, and searches by a single matrix multiply.
- `retrieve.py` — the three modes and the RRF fusion.

Path tokenisation turned out to be load-bearing. An endpoint's address is one opaque token to a computer, so `/chat.postMessage` would never match a search for "post message". Splitting addresses into words —

```
/repos/{owner}/{repo}/issues -> repos owner repo issues
/chat.postMessage             -> chat post message
```

— is the only reason Slack is findable at all, because **Slack's specification contains no descriptions whatsoever**. The address is the only text there is.

3.5 `workflow/` — the model and the gate

- `dag.py` — Pydantic models shared by the planner, the compiler and the executor. Every model forbids unknown fields; silently accepting an invented key is exactly how fabricated structure would slip through.
- `schema_match.py` — JSON Schema path resolution and type compatibility.
- `compiler.py` — the seven checks, and the compiled output.

The seven checks

#	Check	Rejects
1	Structural	cycles, missing or multiple triggers, unreachable steps

#	Check	Rejects
2	Existence	endpoint IDs that resolve to nothing, or to deprecated rows
3	Agreement	a method or path that contradicts the stored record
4	Parameters	required path/query/header parameters with no source
5	Body	required request-body fields with no source
6	Compatibility	upstream output that does not fit downstream input
7	Policy	unsupported authentication, unlisted providers, ungated destructive actions

Rejections are **structured objects**, not sentences:

```
{
  "code": "schema_incompatible",
  "node_id": "n3",
  "field": "message.text",
  "expected": "string",
  "actual": "object",
  "hint": "map issue.title into message.text"
}
```

That single decision pays twice: the same payload renders the interface *and* feeds the model's repair loop.

3.6 agent/ — the language-model layer

- `llm.py` — one narrow interface (`complete_json`) with Gemini and OpenAI implementations and a stub for tests.
- `intent.py` — a sentence becomes a typed requirement.
- `planner.py` — retrieval orchestration, the candidate allowlist, and the bounded repair loop.

The two guards, and why order matters

Guard one — the candidate allowlist. The model may only name endpoint IDs that *this request's searches actually returned*. Anything else is refused **before the database is consulted**.

This is stronger than it first appears. A test confirms that a **real endpoint which exists in the database but was not retrieved for this goal** is still refused. Existing is not enough. That closes the case where a model recalls a genuine endpoint from training data and uses it out of context.

Guard two — the compiler. Anything surviving guard one still faces all seven checks.

The repair loop

When the compiler refuses a plan, the structured issues go back to the model verbatim, up to three times, then it gives up rather than looping. Candidates are presented *with* their required fields and their available response fields, because a model cannot supply a parameter it was never told about.

3.7 execution/ — making it real

- `credentials.py` — Fernet-encrypted storage, decryption at the moment of use, automatic OAuth refresh, append-only audit.

- `oauth.py` — the authorisation-code flow.
- `guards.py` — the egress guard: scheme allowlist, blocked hostnames, private address rejection on **every** resolved address.
- `policy.py` — which providers may receive a real request.
- `executor.py` — request construction, retries, failure classification.
- `runner.py` — ordering, checkpointing, resumption.
- `worker.py` — the polling worker process with graceful shutdown.

The ordering that is the whole reliability story

```
run the node -> write its result -> only then mark it done
```

A crash between the call and the write replays the node, and the idempotency key makes that safe. A crash between the write and the mark also replays, and the checkpoint short-circuits it. What must never happen is marking a node done before its result is durable — that loses work silently, which is worse than doing it twice.

The failure taxonomy

Every failure is exactly one category, and the category alone decides retry behaviour.

Category	Retryable	Cause
<code>validation</code>	no	malformed request, 4xx
<code>auth</code>	no	missing, expired or insufficient credential
<code>rate_limit</code>	yes, with backoff	429
<code>network</code>	yes	timeout, DNS, reset
<code>provider</code>	yes	5xx
<code>data_mapping</code>	no	a mapped value was absent at runtime
<code>policy</code>	no	blocked by the egress guard

3.8 `api/` and `web/`

One FastAPI application, 26 routes. The frontend is Next.js with five pages, and its server-side route handlers act purely as a **backend-for-frontend**: they attach session context and proxy. No business logic lives in the frontend.

Part IV — Design decisions and their reasoning

4.1 One backend language

Decision: Python throughout, rather than splitting between Go and Python.

Reasoning: the Pydantic models the planner emits, the validator rejects and the executor consumes are *the same classes*. A split-language backend defines those contracts twice and lets them drift. Since the project's entire thesis is deterministic validation of a shared contract, that is the single biggest correctness lever available.

The counter-argument, addressed: the execution engine is a concurrency problem, which is conventionally Go's territory. But the workload is I/O bound — a worker waits on provider responses — so one event loop holds many calls in flight, and the GIL never binds. Parallelism comes from more processes, which is how you would scale a Go service too.

4.2 SQLite instead of PostgreSQL, OpenSearch, pgvector and Redis

Decision: one file, no services.

Reasoning: at a 7-day budget, every hour of infrastructure is an hour not spent on the validator. SQLite's FTS5 provides *genuine* BM25, so nothing is lost lexically. Brute-force cosine over 15,504 vectors is a single matrix multiply.

The unexpected benefit: anyone can clone the repository and run it. A portfolio project that requires four services to demonstrate is a worse portfolio project.

The honest cost: this does not prove the system works at 50,000 endpoints on a shared database, which the original specification targets.

4.3 Local embeddings rather than a hosted API

Decision: run a small ONNX model locally.

Reasoning: forced, then vindicated. The hosted free tier allows **1,000 embedding requests per day** and the corpus needs 15,504 — eleven days of waiting. Local inference has no quota and no per-run cost.

Vindicated because it also made the benchmark reproducible: the same input always produces the same vectors, on any machine, with no key.

The cost: about 4 texts per second on the test machine, so roughly 45 minutes for a full corpus. Acceptable for a one-time, resumable job.

4.4 A hand-written queue rather than Celery

Decision: a database job table with leased claiming.

Reasoning: the project's reliability claims are checkpoint-before-acknowledge, resumption, and safe replay. Task frameworks own the acknowledgement lifecycle — precisely the part that must be explicit here. "How do you handle a worker crash after the external call succeeded but before acknowledgement?" is a question you want to answer with your own code.

4.5 Compile to an intermediate representation

Decision: validation produces a stored, immutable artefact that execution reads.

Reasoning: it makes runs reproducible, makes validation a hard gate rather than an advisory pass, and keeps the language model out of the execution path entirely. The executor never sees the original plan.

4.6 Opaque endpoint identifiers

Covered in §3.2. Restated because it is easy to miss: **an unguessable identifier format is a security control**, not a formatting choice.

4.7 Structured validation errors

Decision: rejections are typed objects with a code, a field, expected and actual values, and a hint.

Reasoning: two consumers need the same information — the human reading the screen and the model attempting a repair. A free-text message is a dead end for both. This decision, made on day four, is what made the day-five repair loop straightforward.

4.8 The failure taxonomy drives retries

Decision: classify first, then let the category decide.

Reasoning: it puts the retry policy in one table instead of scattered conditionals, and the same classification feeds the metrics and any future recovery agent.

4.9 Checking the response body, not only the status code

Decision: a `2xx` response whose body reports failure is a failure.

Reasoning: Slack answers `200 OK` with `{"ok": false, "error": "..."}` and it is not alone. Trusting the status code means the automation reports success while silently doing nothing — the exact failure this project exists to prevent.

4.10 The execution allowlist is separate from credentials

Decision: two independent gates — *can* we authenticate, and *may* we call.

Reasoning: they answer different questions, and some APIs need no authentication at all, so a credential cannot be the only thing between a plan and an outbound request.

Part V — Decisions that were wrong

This is the most valuable part of the report. Each of these passed review at the time, shipped, and was later found to be wrong by evidence.

5.1 The corpus was twelve copies of GitHub

The decision: keep one specification per provider, because APIs.guru publishes roughly twenty near-identical GitHub documents (the public API plus every Enterprise Server release, each about 6 MB).

Why it was wrong: `googleapis.com` is not twenty copies of one API. It is **281 entirely different products** — Gmail, Calendar, Drive, Sheets. The rule kept one and discarded 280.

How it was found: a user typed *"whenever I get a Gmail, send me a WhatsApp message"* and the system produced a workflow using Box email aliases and BulkSMS. Investigation showed Gmail was not in the database at all.

The fix: group by provider **and API title**. Every GitHub edition publishes the title "GitHub v3 REST API"; Gmail and Calendar publish different titles.

Impact: distinct APIs visible went from **676 to 2,080**; the corpus went from 10,563 to 15,504 endpoints. **Roughly two thirds of the available catalogue had been invisible.**

The lesson: a deduplication rule encodes an assumption about what "the same thing" means. That assumption was true for GitHub and catastrophic for Google, and nothing in the code recorded which case it was designed for.

5.2 The planner was instructed to guess

The decision: this line in the planning prompt —

"If no candidate fits, use the closest one rather than inventing an identifier."

Why it was wrong: it was written while concentrating on a different danger — stopping the model fabricating identifiers. In closing that door it opened a worse one: it explicitly instructed the model that when nothing fits, **substitute the nearest thing**. So asked for Gmail, it found no Gmail, took the closest email-shaped endpoint, and built something that passed all seven checks.

The fix: a third option. The model may now answer:

```
{"cannot_satisfy": {"reason": "...", "missing": ["gmail", "whatsapp"]}}
```

with the instruction explaining why: *a plan built from the nearest available endpoints is worse than no plan — it looks correct, passes every structural check, and does something nobody asked for.*

The lesson: "always produce an answer" is a failure mode, not a feature. An honest refusal is a legitimate output.

5.3 Reference depth counted the wrong thing

The decision: cap reference resolution at depth 8 to break cycles.

Why it was wrong: the counter incremented on **every structural nesting level**, not only when following a reference. A response schema is routinely eight levels deep before any reference appears, so the limit fired mid-object and replaced real content with a stub. Google Calendar's event `start` — which has `date`, `dateTime` and `timeZone` — was stored as:

```
{"properties": {"type": "object"}, "type": {"type": "object"}}
```

This corrupted schemas across all 15,504 endpoints, silently.

How it was found: the planner reported that Calendar events do not return a `start` field. That claim was checked against the specification, which plainly showed otherwise.

The fix: three separate limits counting three different things — reference hops (cycles), structural depth (pathological documents), and total emitted nodes (size).

The follow-on: removing the accidental truncation caused memory to reach **5.9 GB**, because `seen` prevents *cycles* but not *repetition* — sibling properties each re-expand the same referenced schema in full. Hence the third limit, a shared node budget.

The lesson: one counter guarding three different concerns will be wrong for at least two of them.

5.4 The service name was deleted before searching

The decision: the intent prompt said *"do not name specific companies unless the request does"*, with the example:

"Slack me" becomes "post a message to a channel"

Why it was wrong: the rule was correct and the example contradicted it — the example deletes "Slack".

Models follow demonstrations more reliably than prose. So *"whenever I get a Gmail, send me a WhatsApp message"* became `trigger: "receive a new email"`, `actions: ["send a message"]`, discarding the single most distinguishing term in the request and leaving retrieval hunting for a generic operation across 120 providers.

Measured effect of the fix:

Query	Gmail or WhatsApp retrieved?
"receive a new email"	no
"receive a new email in Gmail "	yes, rank 2

Query	Gmail or WhatsApp retrieved?
"send a message"	no
"send a WhatsApp message"	yes, rank 3

The lesson: in a prompt, an example that contradicts a rule silently overrides it.

5.5 Every automation was forced to have an event

The decision: the requirement schema always required a `trigger`.

Why it was wrong: many useful automations have no event — *"read my latest email and send it on"* is a sequence performed on request. Forced to produce a trigger, the model invented `"receive a new email in Gmail"` for a goal that had none, and the planner then refused because no event endpoint existed.

The fix: `trigger: "manual"`, and the planner told that a trigger node is the workflow's *entry point*, not necessarily an event.

5.6 The model could not name a field it was never shown

The decision: show the model a preview of each candidate's response fields — top-level names plus one level of nested *objects*.

Why it was wrong twice:

1. It never looked **inside arrays**. Gmail's list endpoint returns `{messages: [...]}`; naming a message needs `messages[].id`, which was invisible. The model refused, correctly reasoning from what it could see.
2. The nested budget was **four fields**, taken in specification order. Google Calendar events have forty. The four shown were `anyoneCanAddSelf`, `attachments`, `attendees`, `attendeesOmitted` — and `summary` and `start` were hidden.

The fix: show array item fields with the `[]` marker the compiler already understands, raise the budget to forty, and **rank** fields so useful ones survive.

The lesson: when a model says something is impossible, check whether you told it the truth. Both times the model's reasoning was sound and the input was not.

5.7 The validator and the executor disagreed

The decision: support array syntax in dotted paths.

Why it was wrong: two separate implementations. The compiler's resolver accepted both `messages[]` and `messages[0]`; the executor's reader understood only `messages[]`. A plan containing `n1.messages[0].id` **passed every check and would have failed at runtime**.

That is the worst possible bug for this project, because the validator's entire purpose is preventing exactly that.

The fix: both share one compiled pattern, and a test takes every path the validator accepts and asserts the executor can read it.

The lesson: two implementations of one concept will drift unless something forces them together.

5.8 A preview that refused to preview

The decision: enforce the execution allowlist in the executor.

Why it was wrong: the check ran *before* the dry-run branch, so a dry run — which sends nothing — failed with a policy error. The request had been built perfectly and was rejected for a rule about *sending*. Worse, it broke the feature exactly when it is most useful: previewing a workflow for a provider not yet enabled.

This was the same mistake as the credential check fixed a day earlier, **two lines above it in the same function**.

The fix: a dry run reports what would stop it rather than failing.

5.9 A hard-coded provider list

The decision: a curated list of ten providers on the credentials page.

Why it was wrong: the list *was the gate*, so the other 110 indexed providers were unreachable — Telegram, Notion, Zoom, Bitbucket, Jira. The same failure of imagination as §5.1: building for the handful in mind rather than the corpus that exists.

The fix: drive the page from the database, and surface metadata that had been ingested since day one and never used — 62 of 120 providers carry a documentation link. The curated list became an enhancement layer.

5.10 Two misleading classifications

Found while testing the fix for §5.9:

- **GitHub reported "declares no authentication"**. True of the file — GitHub's official OpenAPI document genuinely omits `securitySchemes` — and misleading about the service, which obviously needs a token.
- **Slack was classified "impossible"**. Its specification declares `oauth2`, so the rule marked it unconnectable, while the project's own setup guide told the user to paste an `xoxb-` bot token. Straightforwardly self-contradictory.

The fix: OAuth in a specification does not mean a sign-in flow is the only way in. Only two things are structurally impossible — a token embedded in the URL, and a service with no public host.

5.11 A dead column, and the tests that hid it

`providers.allowlisted` existed from day one, was written at ingestion, and was **never read**. Every check consulted the environment file instead, so the only way to permit a provider was to edit a file and restart.

When the column was finally honoured, **three tests failed** — because the test fixture set `allowlisted=True` on every provider. Harmless while the column was dead; silently wrong the moment it mattered.

The lesson: a fixture that sets a field the code ignores is a landmine.

5.12 What these have in common

Not one of them was a crash. Every component reported success. Ingestion said it ingested, search said it found, the validator said all seven checks passed. The system was confidently, silently wrong.

The tests could not have caught them, because every fixture contained the endpoints the author expected.

That is the honest limit of a test suite: it proves the code does what you think, never that what you think is right.

Every one was found by a person typing a real request and reading the answer.

Part VI — The research that informed the work

BM25 rather than embeddings alone. API queries are full of exact tokens — `repos` , `POST` , `chat.postMessage` — where lexical matching is strong, while semantics cover paraphrase. The measurements in Part VII confirm neither wins alone.

Reciprocal Rank Fusion. Chosen over score normalisation because BM25 scores and cosine similarities are not comparable, and RRF needs exactly one parameter.

Structured output with strict schemas. Provider JSON modes vary in dialect and strictness, so the response is parsed into a Pydantic model with `extra="forbid"` regardless of what the provider promises.

Bounded self-repair. Feeding validation errors back is effective; letting it loop is not. Three attempts, then an honest failure.

At-least-once plus idempotency. The standard distributed-systems answer, chosen over pretending to have exactly-once semantics.

Cross-encoder reranking was investigated and *not* adopted, because the latency budget is already nearly spent (Part VII). It is the top candidate for the next round of work.

Part VII — Operating findings and measured numbers

All figures measured on 29 August 2026 against the full 15,504-endpoint corpus. Reproduce with `uv run python bench/run_bench.py`.

7.1 Retrieval quality — 30 labelled queries

Mode	P@5	R@20	MRR	NDCG@10	p50	p95	queries with no hit
bm25	0.133	0.489	0.372	0.295	16 ms	32 ms	8
vector	0.153	0.578	0.409	0.324	376 ms	468 ms	6
hybrid	0.207	0.598	0.485	0.394	458 ms	505 ms	6

Hybrid wins every quality metric — NDCG@10 is 34% above BM25 alone.

On reading these: P@5 divides by 5 even when a query has one correct answer, so 0.2 is the ceiling for such queries. Compare the modes, not the distance from 1.0. **R@20 matters most operationally**, because the planner only ever sees the top 20 — anything below is invisible to the rest of the system.

7.2 The corpus grew and the scores fell

An earlier run against 10,563 endpoints scored hybrid NDCG@10 at **0.412**. After the fix in §5.1 expanded the corpus by 47%, the same queries score **0.394**.

That is expected and worth stating plainly: **a larger haystack is a harder retrieval problem**. Quoting the older, better number beside the newer, larger corpus figure would be quietly false.

7.3 Two experiments, one of which failed

Synonym expansion — kept. A single probe query suggested it hurt. Measured across all 30, it helps: NDCG@10 0.248 → 0.301 on the earlier corpus, with two fewer complete failures. **The anecdote pointed the wrong way**, which is the whole argument for building the benchmark before tuning.

Symmetric synonyms — rejected. The table declares `issue -> bug` without `bug -> issue`. Synonymy is symmetric by definition, so deriving the reverse looks like an obvious correctness fix. Measured, it was worse: NDCG@10 0.301 → 0.241, complete failures 8 → 11. It fixed one query and broke five, because every added term dilutes the signal and symmetrising produced chains like `user -> member -> account`.

Reverted, with the measurement recorded in the source so it is not retried.

7.4 Planning cost

Sentence to compiled workflow	~20 seconds
-------------------------------	-------------

Tokens, accepted first attempt	2,582
Tokens, one repair round	5,850

7.5 Reliability

Re-running a completed execution issues **zero** repeat HTTP calls. A node that failed can be cleared and retried while completed nodes are not re-executed.

7.6 What has actually been executed

A distinction worth drawing precisely, because "it executes" is easy to claim and easy to overstate. Taken from the execution tables:

Workflows saved	12
Executions recorded	18
Of those, real rather than dry-run	9
Real executions that succeeded end to end	2
Live HTTP 200 responses received	4
Credentials stored	0

So real requests have left the process, reached live services, and returned success. The executor is not merely architecturally plausible.

But no authenticated call has ever been made against a live provider. With no credential stored, the path that decrypts a secret, applies it to a request and has a real service accept it exists and is covered by tests using a mocked transport — never against the real thing.

That is the cheapest remaining gap in the project and the one most worth closing: one stored token and one workflow run converts an architectural claim into an observed fact.

7.7 Ingestion

Parallel downloading with an 8 MB cap: **118 specifications in about 40 seconds**, against 4 in 15 minutes sequentially. Local embedding runs at roughly **4 texts per second**; forcing 22 ONNX threads on a 22-core machine made it *slower* through contention.

Part VIII — Running and extending it

8.1 From a clean clone

```
uv sync --extra dev
cp .env.example .env

uv run engine fetch --limit 160      # download specifications
uv run engine ingest                 # parse into the database
uv run engine index                  # build the keyword index
uv run engine embed                  # build vectors locally (~45 min, resumable)
uv run engine stats
```

8.2 Using it

```
uv run engine search "send a message to a channel" --mode hybrid
uv run engine plan "when a github issue opens, post it to slack"
uv run engine run <workflow_id> --dry-run
uv run pytest -q
uv run python bench/run_bench.py
```

Web interface:

```
uv run uvicorn engine.api.main:app --port 8000      # API and docs at /docs
cd web && npm install && npm run dev                 # http://localhost:3000
```

8.3 Extending it

A new provider is a data operation, not a code change: ingest its specification and it becomes searchable and plannable.

A new node type — add it to `dag.py`, teach `compiler.py` how to validate it, and `runner.py` how to run it.

A new authentication scheme — extend `apply_auth` in `credentials.py` and the `SUPPORTED_SCHEMES` set in `parse.py`.

A new LLM provider — implement `complete_json` in `llm.py`. Nothing above that module knows which provider is in use.

Part IX — Honest limitations

Validation proves an automation *can* run, never that it does what you meant. `title`, `body` and `user.login` are all strings; no checker knows which was intended. In the project's own flagship demonstration the model mapped the issue *body* where the *title* was asked for — a perfectly valid workflow that was not the request. This is why plans are reviewable before being enabled.

No authenticated call has been made against a live provider. Unauthenticated requests have genuinely succeeded (§7.6), and the credential path is covered by tests against a mocked transport, but the combination of a real token and a real service has not been demonstrated.

No triggers. Nothing fires by itself. There are no webhooks, no polling, no scheduler, so *"whenever I receive an email"* cannot work regardless of credentials. This is the largest single gap.

The schema checker models a documented subset. `oneOf`, `anyOf`, `allOf`, cross-document references, `format` and numeric ranges are out of scope and return *unknown*, which is allowed through. It is a gate against provably wrong plans, not a proof of correctness — refusing everything unmodellable would reject most real specifications.

Hybrid search exceeds its latency target. 505 ms at p95 against a 500 ms goal, almost entirely local query-embedding time rather than search itself.

Thirty labelled queries is a small sample. Every retrieval figure here rests on them. Differences of a few points should not be over-read.

Providers that put the token in the URL cannot execute. Telegram's address is `https://api.telegram.org/bot{token}`. Requests are built from indexed metadata and the engine does not substitute into the host.

OAuth is implemented for Google only, and requires the user to register their own client.

One machine, one process. No horizontal scaling has been demonstrated, and the corpus is 15,504 endpoints against a specification target of 50,000.

Part X — Glossary

Term	Meaning
API	A way for one program to ask another to do something
At-least-once	A delivery guarantee: a job arrives, possibly more than once
Backoff	Waiting longer after each failed retry
BM25	A classical keyword-ranking algorithm weighting rare words higher
Checkpoint	Durably recorded progress, so a crash resumes rather than restarts
Compiled IR	The immutable validated form of a workflow that execution reads
Corpus	The whole indexed collection being searched — here, 15,504 endpoints parsed from OpenAPI specifications
Cosine similarity	The angle between two vectors, used to compare meanings
Cross-encoder	A reranking model that reads query and document together
DAG	Directed acyclic graph — steps with arrows that never loop back
Dry run	Executing a workflow with every outbound request suppressed, to prove the wiring without messaging anyone
Egress guard	The check applied before any outbound request: allowed scheme, blocked hostnames, no private addresses. The defence against SSRF
Embedding	A list of numbers representing the meaning of text
Endpoint	One specific operation an API offers, a method plus a path
Fernet	A symmetric encryption scheme used here for stored credentials
FTS5	SQLite's full-text search module, which implements BM25
Hallucination	A model stating a fabricated endpoint or field in the same confident tone it uses for correct ones
HTTP	The request-and-response protocol the web runs on, and the way every API call in this project is made
Idempotency key	A value making a repeated request safe to send
Jitter	Randomness added to retry delays to avoid synchronised retries
JSON Schema	A standard description of the shape of JSON data
Large language model (LLM)	A program that predicts likely text. Excellent at turning phrasing into structure; it does not look facts up
MRR	Mean reciprocal rank — how near the top the first correct answer is
NDCG	Ranking quality with a discount for lower positions
Node	One step in a workflow — a trigger or an action. Used interchangeably with "step"

Term	Meaning
OAuth 2.0	An authorisation flow that issues a token after user approval
ONNX	A portable format for running models without a deep-learning framework
OpenAPI	A machine-readable description of an entire API
p95	The value 95% of measurements fall under
Payload	The body of an HTTP request or response. A GitHub issue payload runs to roughly 200 lines
Precision@K	Of the top K results, the fraction that were correct
Prompt	The text handed to the model, here including the retrieved candidates with their fields
Pydantic	A Python library that validates data against typed models
Queue	A waiting list of jobs, so accepting work and doing it are separate and a crash loses nothing
Recall@K	Of all correct answers, the fraction reaching the top K
\$ref	A pointer inside a specification to a schema defined elsewhere
Refresh token	A durable credential used to obtain new access tokens
RRF	Reciprocal rank fusion — merging ranked lists by position
SSRF	An attack pointing a URL-fetching system at internal infrastructure
Swagger 2.0	The predecessor of OpenAPI 3, still widely published
Token (LLM)	The unit a model reads and writes text in, and the unit its cost is counted in. A compiled plan costs ~2,582
Trigger	The step that starts a workflow, such as "a GitHub issue was opened". Every workflow has exactly one
WAL	SQLite's write-ahead logging mode, allowing concurrent readers
Webhook	An HTTP request a service sends you when something happens, instead of you polling it repeatedly
Worker	The process that takes jobs off the queue and runs them
Workflow	An automation expressed as data rather than code: steps plus the dependencies between them

End of report.